

SkyGuardX-AI

Real-Time Space-to-Earth Decision Intelligence

End-to-End Master Project Specification

Buildathon positioning: Real-Time AI for Critical Decisions

A prototype decision platform that ingests real-time satellite, space-environment and Earth-observation signals; predicts risk; models cascading consequences across infrastructure and people; runs what-if scenarios; and produces explainable, human-approved recommendations.

CORE THESIS

We do not stop at detecting an event. SkyGuardX-AI estimates what happens next, quantifies downstream impact, compares response options, and recommends the action with the best risk–impact trade-off.

Document	Version	Status	Primary scope
Master Project Specification	1.0	Buildathon-ready design	MVP → advanced roadmap

Important prototype assumption: orbital telemetry, disaster observations and live event streams may be simulated or replayed from historical/open datasets during the hackathon. The system must visibly distinguish simulated data from live operational data.

1. Executive Summary & Overview

Project title: SkyGuardX-AI — Real-Time Space-to-Earth Decision Intelligence.

SkyGuardX-AI is a decision-support platform for time-critical events where a signal detected in or from space can create consequences on Earth. It combines streaming data, machine learning, geospatial analytics, scenario simulation and generative AI explanations around a reliable transactional processing layer aligned with IBM Z.

In plain English, the platform answers five questions: **What is happening? How risky is it? What could happen next? What happens under alternative actions? What should an operator do now?**

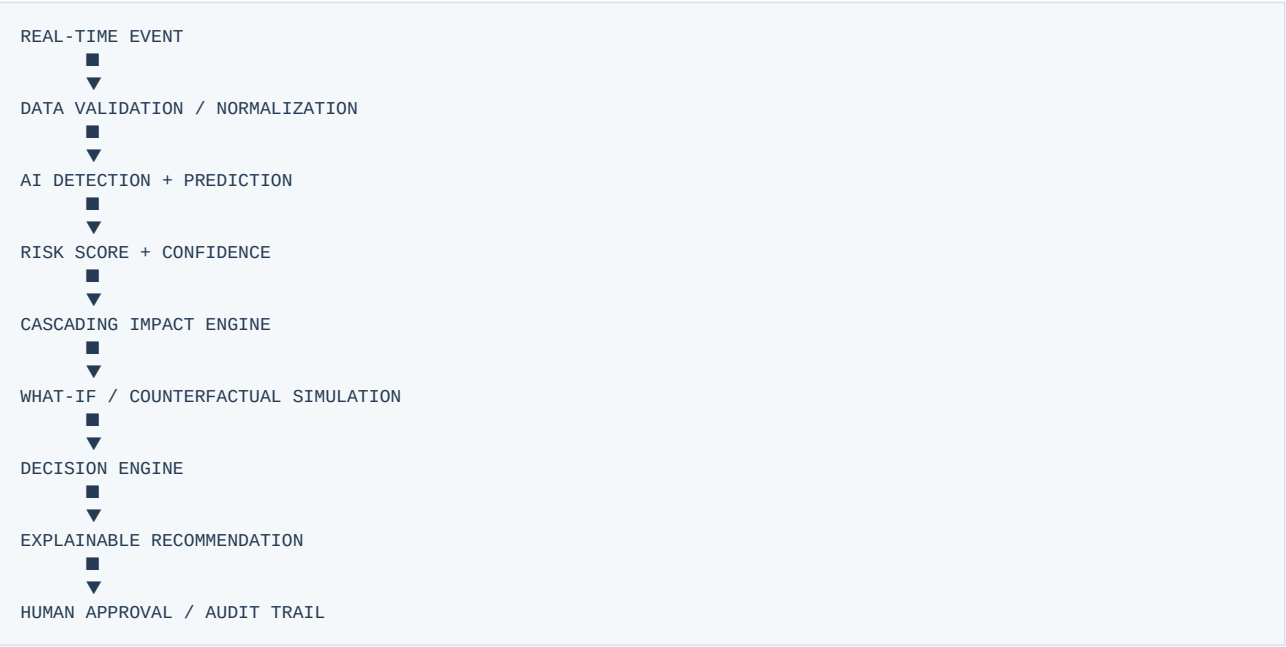
VALUE PROPOSITION

Convert fragmented real-time observations into an explainable chain from signal → prediction → cascading impact → what-if simulation → recommended action. This shortens the gap between insight and intervention.

1.1 Target audience

Persona	Need	SkyGuardX-AI outcome
Emergency / disaster operations analyst	Rapidly understand developing hazards	Risk map, affected assets, resource recommendation
Satellite / mission operator	Detect spacecraft anomalies and conjunction risk	Telemetry health, collision risk, recovery options
Infrastructure coordinator	Estimate service disruption	Road, hospital and population impact
Incident commander	Choose among response strategies	What-if comparison and prioritized action
Data/AI engineer	Monitor model and pipeline quality	Data lineage, confidence, drift and observability

1.2 North-star workflow



2. Problem Statement & Motivation

Critical-event systems often have data, dashboards and alerts, but the information is fragmented. A satellite operator may see telemetry anomalies while an emergency team separately sees weather, maps, traffic or hospital capacity. The operational gap is not only detection—it is connecting a new event to downstream consequences and selecting a response quickly.

2.1 Exact problem

PROBLEM STATEMENT

When high-volume space and Earth data changes rapidly, operators need a trustworthy way to transform events into near-real-time risk estimates, understand cascading impact, compare response options and receive an actionable recommendation without hiding uncertainty or removing human accountability.

2.2 Why conventional approaches fall short

Gap	Typical limitation	SkyGuardX-AI response
Detection without action	Alerts identify anomalies but leave the operator to manually investigate consequences.	Decision engine connects prediction to response options.
Siloed data	Telemetry, Earth observation, roads and facilities may be analyzed separately.	Unified event and entity model links space and Earth context.
Late insight	Batch analytics can be useful after an event but too slow for critical decisions.	Streaming/replay pipeline evaluates events as they arrive.
Black-box recommendations	A model score alone does not explain why an action is preferred.	Feature-level explanations, evidence and scenario comparisons.
Single-outcome prediction	One forecast hides uncertainty and alternative futures.	What-if simulation exposes multiple possible outcomes.
AI overreach	Fully autonomous actions can be unsafe for mission-critical systems.	Human-in-the-loop approval, policy constraints and audit logs.

2.3 Root causes

- **Data fragmentation:** operational context exists across heterogeneous systems and schemas.
- **Temporal pressure:** the value of a prediction decreases as the response window closes.
- **Cascading effects:** an initial hazard can alter roads, demand, capacity and resource availability.
- **Decision complexity:** the best action balances risk reduction, cost, fuel, accessibility and operational constraints.
- **Uncertainty:** observations can be noisy, incomplete or delayed, so confidence and data quality must be first-class signals.

2.4 Why now

Modern streaming infrastructure, geospatial analytics, open Earth-observation datasets, compact ML models and generative AI make it feasible to prototype a system that combines detection, prediction, simulation and natural-language explanation. IBM Z strengthens the architecture when critical transactional data and inference need to remain close to enterprise workloads.

3. Solution & Product Vision

SkyGuardX-AI is designed as a **decision intelligence layer**, not a replacement for domain control systems. It receives events and context, computes risk and consequences, compares feasible actions, and presents evidence to an authorized human.

3.1 Key capabilities

Capability	What it does	MVP priority
Real-time event ingestion	Accept telemetry, orbital events, weather/disaster observations and operational updates.	Must
Space intelligence	Satellite health/anomaly scoring and conjunction/collision-risk estimation.	Must
Earth impact analysis	Estimate affected area, population, roads and selected critical facilities.	Must
Cascade engine	Propagate hazard effects into infrastructure and human-impact indicators.	Must
What-if simulation	Compare outcomes under different interventions or hazard intensities.	Must
Decision engine	Rank actions using risk, impact, resource and policy constraints.	Must
GenAI explanation	Turn structured evidence into an operator-readable rationale.	Enhancement
Continuous learning	Monitor drift, retrain and compare candidate models.	Future

3.2 Differentiators

- **Space-to-Earth linkage:** connects space-derived signals to operational consequences instead of treating satellite data as the final product.
- **Cascade-first design:** explicitly models second-order effects such as road accessibility, emergency demand and facility load.
- **Decision over detection:** the product's main output is a recommended action with evidence, not simply an anomaly label.
- **What-if by design:** operators can change assumptions and see how projected outcomes move.
- **Responsible autonomy:** AI proposes; policy and authorized humans approve high-consequence actions.
- **IBM Z alignment:** positions real-time inference and transactional data processing near critical enterprise workflows.

3.3 Primary use cases

Use case	Trigger	Decision output
Satellite health	Telemetry deviation	Continue / inspect / safe-mode recommendation
Conjunction risk	Close approach detected	No maneuver / maneuver A / maneuver B with risk–fuel trade-off
Wildfire response	Thermal/spatial anomaly + weather	Resource staging, route protection, evacuation preparation
Flood response	Water/flood extent change	Road closure, ambulance relocation, facility load mitigation

4. Project Roadmap & Implementation Phases

The roadmap deliberately prioritizes one complete vertical slice before adding breadth. The recommended first scenario is **satellite/remote-sensing event** → **wildfire or flood** → **infrastructure impact** → **what-if** → **recommendation**.

4.1 Phase 1 — MVP / Foundation

Workstream	Deliverables	Acceptance criterion
Data	Event schema, replayable stream, sample satellite/earth/weather datasets	Events arrive with timestamp, source, entity ID and quality status
Space	Satellite/debris map, telemetry anomaly model, basic conjunction risk	Risk score produced for each event
Earth	One disaster model (wildfire or flood), geospatial overlay	Affected geometry and exposure calculated
Impact	Population, roads and 1–2 critical facilities	Impact score updates after event
Decision	2–3 response actions with deterministic constraints	Ranked recommendation with evidence
UI	Command-center dashboard and live event timeline	Judge can follow an event end-to-end
IBM Z	Transactional data/inference integration or representative deployment boundary	Architecture shows meaningful Z role and demonstrable transaction path

4.2 Phase 2 — Feature expansion

- Add the second disaster type and richer satellite health signals.
- Introduce model ensembles, SHAP/explanation views and calibrated confidence.
- Add traffic prediction, hospital demand and emergency-resource constraints.
- Add natural-language scenario queries such as “What if the affected area grows by 20%?”
- Add model/data observability, event replay controls and incident audit timeline.

4.3 Phase 3 — Advanced optimization, scale & future scope

- Optimization-based resource allocation and route planning.
- Streaming feature stores and low-latency inference at larger event rates.
- Federated/edge sensing where operational constraints require it.
- Multi-agent simulation for competing response strategies.
- Online drift detection, retraining pipelines and champion/challenger evaluation.
- Multi-region disaster correlation and long-horizon resilience planning.

4.4 Definition of Done for the buildathon

DEMO-READY

A single event must be traceable from ingestion to AI score, impact propagation, scenario comparison and human-readable recommendation. Every displayed metric must identify whether it is measured, modeled, simulated or derived.

5. Technology Stack & Infrastructure

The stack should optimize for speed of implementation while preserving a credible production architecture. Specific IBM services should be selected based on the environment made available by the competition; the design below separates stable concepts from implementation-specific products.

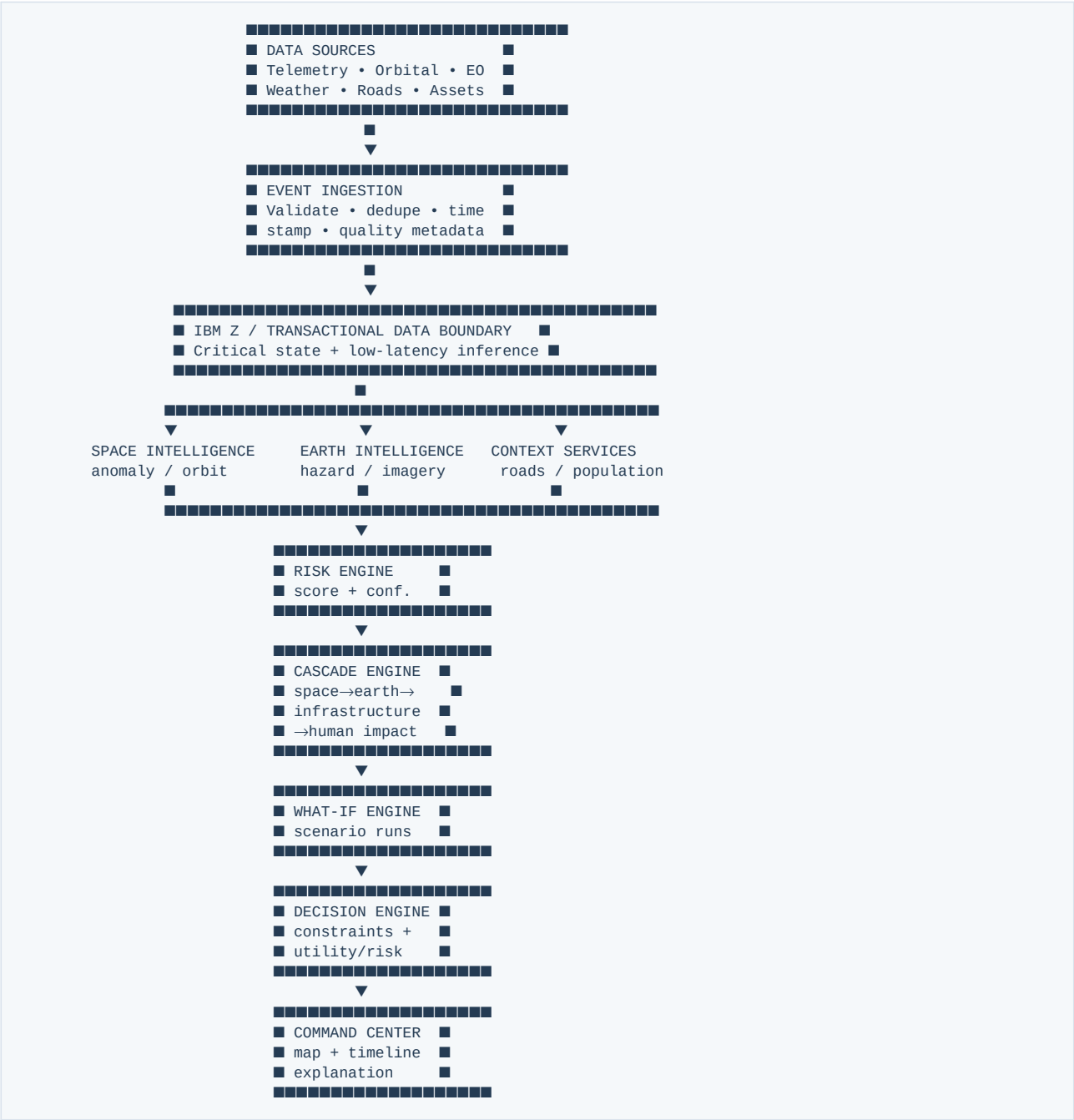
Layer	Recommended technology	Purpose	Why this choice
Frontend	React + TypeScript; MapLibre/Leaflet	Command center, map, event stream, scenario UI	Fast iteration; strong geospatial visualization ecosystem
API	Python FastAPI	REST APIs and orchestration	Excellent ML integration and typed API contracts
Streaming	Kafka / IBM Event Streams equivalent	Event ingestion and replay	Durable event-driven architecture
Transactional layer	IBM Z + enterprise transaction/data services available in contest	Critical operational data and inference boundary	Aligns directly with Transactional AI theme
Operational DB	PostgreSQL + PostGIS	Entities, events, geospatial relationships	Strong relational + spatial support
Cache	Redis	Hot risk/context data	Low-latency reads
ML	scikit-learn, XGBoost; PyTorch when sequence/CV is justified	Anomaly, risk and forecasting models	Practical, explainable and fast to iterate
Geospatial	GeoPandas, Shapely, raster/vector tooling	Exposure and spatial impact	Mature open ecosystem
GenAI	LLM via approved IBM/cloud endpoint	Explanation and natural-language scenario interface	Keeps generative layer above deterministic analytics
MLOps	MLflow + containerized services	Experiment/model registry	Reproducibility and promotion workflow
DevOps	Docker, CI/CD, Kubernetes/OpenShift if available	Deployment and scaling	Portable deployment model
Observability	OpenTelemetry + Prometheus/Grafana	Latency, errors, model/pipeline health	Critical for real-time systems
Security	OIDC/OAuth2, TLS, RBAC, secrets manager	Identity, authorization and secret handling	Enterprise baseline

5.1 Technology trade-offs

Decision	Chosen direction	Alternative	Trade-off
FastAPI vs. Node-only backend	FastAPI	Express/NestJS	Python minimizes ML service boundary; Node may be better for JS-only teams.
PostgreSQL/PostGIS	Relational + spatial	NoSQL-only	Slightly less flexible for arbitrary documents, much stronger for joins, integrity and geospatial queries.
XGBoost/Isolation Forest	Tabular + anomaly baselines	Deep learning everywhere	Less data-hungry, faster to explain and tune; deep models reserved for justified tasks.
Event-driven pipeline	Kafka-compatible stream	Polling REST APIs	More operational complexity, but better temporal semantics and replay.
LLM for explanation only	Structured evidence → LLM	LLM as predictor	Reduces hallucination risk and preserves deterministic decision logic.

6. System Architecture & Technical Design

6.1 Logical architecture



6.2 Component responsibilities

Component	Inputs	Outputs	Key design rule
Ingestion gateway	Raw events	Validated canonical events	Never trust source schema or timestamp blindly
Event bus	Canonical events	Ordered/replayable streams	Partition by entity/event domain
Feature service	Recent + historical context	Model-ready features	Version feature definitions
Space model service	Telemetry/orbital data	Health/conjunction risk	Expose model version + confidence
Earth model service	Imagery/weather/geospatial	Hazard probability/extent	Track observation time vs processing time

Component	Inputs	Outputs	Key design rule
Cascade engine	Hazard + asset graph	Impact estimates	Make propagation assumptions explicit
Scenario engine	Baseline state + action	Counterfactual outcomes	Never mutate production state
Decision engine	Scenario outcomes + policies	Ranked actions	Hard constraints override learned preferences
Explanation service	Structured evidence	Natural-language rationale	LLM cannot invent source facts

6.3 Data flow

SOURCE → INGEST → VALIDATE → ENRICH → FEATURE BUILD
→ MODEL INFERENCE → RISK SCORE → IMPACT GRAPH
→ SCENARIO SIMULATION → ACTION SCORING
→ EXPLANATION → UI / AUDIT

Every event carries:
event_id | source | entity_id | event_time | ingest_time
schema_version | quality_flag | location | payload | lineage

6.4 Canonical data model

Entity	Important fields	Relationships
Event	event_id, type, source, event_time, ingest_time, payload_hash, quality	belongs to source/entity; may trigger risk
Satellite	satellite_id, orbit_state, mission_state, owner, health_state	has telemetry; participates in conjunctions
SpaceObject	object_id, type, position, velocity, uncertainty	conjunction candidate with satellite
Observation	observation_id, sensor, geometry, timestamp, quality, artifact_uri	supports Earth hazard inference
Hazard	hazard_id, type, probability, geometry, severity, valid_from/to	impacts assets/population
Asset	asset_id, type, location, capacity, status	road/hospital/utility/etc.
Risk	risk_id, subject_id, score, confidence, model_version	derived from event/features
Scenario	scenario_id, baseline_id, assumptions, actions, outputs	compares alternatives
Recommendation	recommendation_id, scenario_id, action, rationale, score, approval_state	linked to audit trail
AuditRecord	record_id, actor, timestamp, action, evidence_refs	immutable decision history

6.5 Suggested relational schema

```

satellite(satellite_id PK, name, mission_state, health_state, updated_at)
telemetry(event_id PK, satellite_id FK, event_time, metrics_json, quality)
space_object(object_id PK, type, orbit_state_json, updated_at)
conjunction(id PK, satellite_id FK, object_id FK, tca, miss_distance, probability)
observation(id PK, source, observed_at, geometry, quality, artifact_uri)
hazard(id PK, type, probability, severity, geometry, valid_from, valid_to)
asset(asset_id PK, type, geometry, capacity, status)
impact(id PK, hazard_id FK, asset_id FK, exposure, accessibility, demand)
scenario(id PK, baseline_id, assumptions_json, created_at)
scenario_result(id PK, scenario_id FK, action, metrics_json, score)
recommendation(id PK, scenario_id FK, action, confidence, rationale_json, state)
audit(id PK, recommendation_id FK, actor, timestamp, event_json)

```

6.6 Event-state semantics

Use **event time** for when something actually happened, **ingest time** for when the platform received it, and optionally **processing time** for when inference completed. This distinction is essential when delayed telemetry or out-of-order observations are replayed.

7. Detailed Implementation Details

7.1 Core module logic

A. Telemetry anomaly detection

- Normalize sensor metrics and retain missingness indicators rather than silently imputing everything.
- Create rolling features: mean, standard deviation, slope, deviation from expected operating envelope and recent change rate.
- Run a baseline anomaly detector (Isolation Forest or robust statistical thresholding).
- For sequential data, optionally add an LSTM/autoencoder only after a strong baseline exists.
- Calibrate the resulting anomaly score into a bounded risk score and attach confidence/data-quality metadata.

B. Conjunction / collision-risk pipeline

- Ingest satellite and space-object state vectors with timestamps and uncertainty.
- Propagate states to a common time horizon using an appropriate orbital propagation method.
- Compute time of closest approach (TCA), miss distance and uncertainty-aware collision probability.
- If using ML, use physically derived features as inputs; do not replace orbital mechanics with an opaque classifier.
- Generate candidate maneuvers subject to fuel, mission and safety constraints.
- Return ranked alternatives rather than one unqualified command.

C. Earth hazard and exposure

- Generate or ingest hazard geometry/probability from remote sensing and contextual data.
- Intersect hazard geometry with population, road and facility layers.
- Estimate exposure and accessibility loss using transparent spatial rules.
- Feed these outputs into the cascade graph rather than directly into the LLM.

D. Cascade engine

```
Hazard
■ affects → road accessibility
■          ■ changes → travel time
■ affects → population exposure
■ changes → emergency demand
            ■ loads → hospital capacity
```

Each edge has:
relationship type | coefficient/rule | confidence | source/version

7.2 Decision engine

Represent each candidate action as a structured object. Calculate an overall utility score from normalized risk reduction, human-impact reduction, resource cost, time-to-effect and operational penalties. Hard constraints—such as prohibited actions, minimum safety thresholds or authorization requirements—must be evaluated before learned preferences.

```
action_score =  
    w1 * risk_reduction  
  + w2 * human_impact_reduction  
  + w3 * response_speed  
  - w4 * resource_cost  
  - w5 * mission_disruption  
  - w6 * uncertainty_penalty  
  
subject to:  
    safety_constraints = TRUE  
    authorization = VALID  
    data_quality >= MIN_THRESHOLD
```

7.3 What-if engine

Scenarios are immutable snapshots containing a baseline state, explicit assumptions and one or more candidate actions. The engine reuses the same models but changes only scenario inputs. This prevents accidental contamination of the live state.

Scenario parameter	Example	Expected output
Hazard extent	+20% flood area	new affected roads/population
Response delay	+15 minutes	new exposure and facility load
Resource placement	ambulance moved to Zone B	new ETA and coverage
Satellite maneuver	burn profile B	new miss probability and fuel use
Weather factor	wind +15%	new wildfire spread probability

7.4 API structure

Method	Endpoint	Purpose
GET	/api/v1/events	Query recent events and filters
GET	/api/v1/satellites/{id}/health	Satellite health and telemetry-derived risk
GET	/api/v1/conjunctions	Current conjunction candidates
POST	/api/v1/scenarios	Create a what-if scenario
POST	/api/v1/scenarios/{id}/run	Run scenario simulation
GET	/api/v1/scenarios/{id}/results	Retrieve scenario outcomes
POST	/api/v1/recommendations/generate	Generate ranked actions from structured evidence
POST	/api/v1/recommendations/{id}/approve	Record authorized human approval
GET	/api/v1/audit/{id}	Retrieve evidence and decision history

Example event payload

```
{
  "event_id": "evt-1042",
  "event_type": "telemetry_anomaly",
  "source": "simulator",
  "entity_id": "SAT-1042",
  "event_time": "2026-09-09T10:41:22Z",
  "quality": {"status": "GOOD", "latency_ms": 180},
  "payload": {
    "battery_voltage": 24.8,
    "solar_output": 0.61,
    "thermal_index": 0.83
  },
  "schema_version": "1.0"
}
```

Example recommendation response

```
{
  "recommendation_id": "rec-77",
  "action": "HOLD_AND_VERIFY",
  "confidence": 0.94,
  "expected_effect": {"risk_reduction": 0.71},
  "evidence": [
    "solar_output_5m_delta",
    "thermal_index_zscore",
    "battery_voltage_deviation"
  ],
  "requires_human_approval": true,
  "model_versions": {"risk": "risk-1.3", "impact": "impact-0.8"}
}
```

7.5 Error handling & edge cases

Edge case	Risk	Mitigation
Out-of-order events	Incorrect temporal inference	Watermarks, event-time windows and late-event policy
Duplicate events	Inflated counts/risk	Idempotency key = source + event_id + version
Missing telemetry	False confidence	Missingness features; confidence penalty; safe fallback
Conflicting sources	Wrong state	Source priority, freshness and reconciliation policy
Stale data	Bad recommendation	TTL checks; display data age prominently
Model unavailable	Decision pipeline stops	Fallback rules/statistical baseline; mark degraded mode
LLM unavailable	Explanation missing	Show structured evidence and deterministic recommendation
LLM hallucination	False rationale	LLM receives only structured evidence; validate citations/fields
Scenario explosion	Latency/cost spike	Limit scenarios, cache shared computations, async execution
Adversarial/bad input	Pipeline poisoning	Schema validation, bounds checks, provenance and quarantine
Unsafe recommendation	Operational harm	Hard policy constraints + human approval

7.6 Design patterns

- **Event-driven architecture:** decouples producers from consumers and supports replay.
- **Strategy pattern:** lets different hazard/risk models implement a common inference interface.
- **Repository pattern:** isolates persistence from domain logic.
- **Adapter pattern:** normalizes external satellite, weather and map providers into canonical schemas.
- **Command pattern:** represents proposed actions as auditable objects rather than direct side effects.
- **Circuit breaker:** prevents repeated calls to degraded external services.
- **Human-in-the-loop:** separates recommendation generation from consequential execution.

8. Security, Scalability & Performance

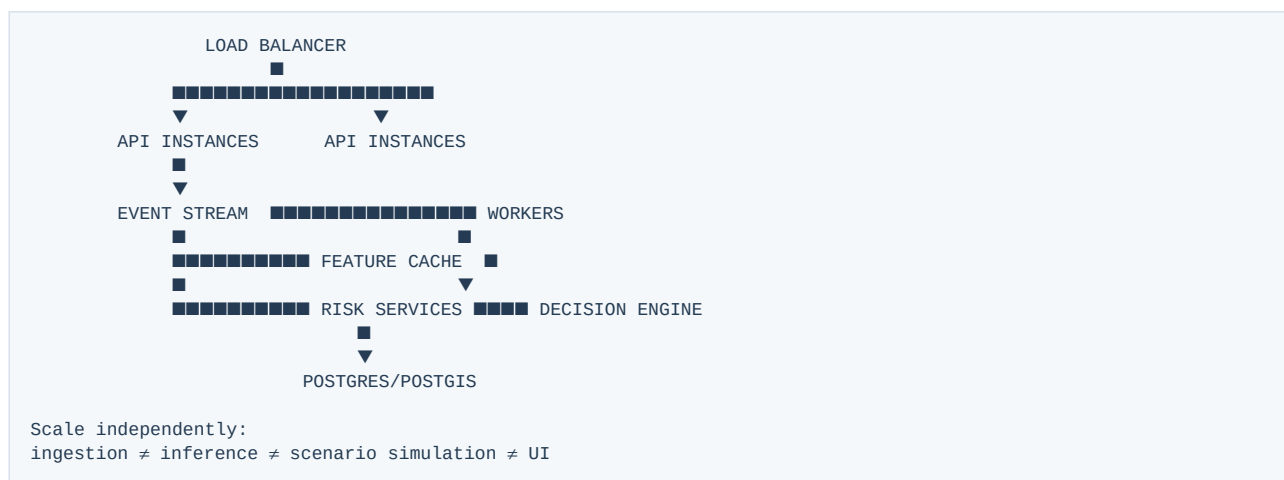
8.1 Security architecture

Control	Implementation	Purpose
Authentication	OIDC/OAuth2 with short-lived tokens	Verify user/service identity
Authorization	RBAC + action-level policy checks	Prevent unauthorized recommendations/approvals
Transport	TLS for service-to-service and client traffic	Confidentiality/integrity
Secrets	Managed secrets store; never in source code	Protect credentials and API keys
Data minimization	Store only required operational attributes	Reduce privacy exposure
Encryption at rest	Database/object-store encryption	Protect stored data
Auditability	Immutable decision and approval records	Reconstruct who/what/why
Model governance	Version model, features, training data and thresholds	Reproducibility
LLM safety	Structured prompts, output schema, grounded evidence	Reduce hallucination and prompt injection

8.2 Responsible AI requirements

- Display confidence and data freshness alongside every critical prediction.
- Never equate probability with certainty; use calibrated language such as estimated risk.
- Provide an evidence panel linking recommendations to model outputs and data sources.
- Keep a human approval gate for high-consequence actions.
- Monitor false positives/false negatives separately because their costs differ.
- Version thresholds and decision policies so historical recommendations remain reproducible.

8.3 Scalability strategy



8.4 Performance targets for the prototype

Metric	Target	Measurement
Event ingestion acknowledgement	< 500 ms p95	Timestamp at gateway
Simple anomaly inference	< 1 s p95	End-to-end event → score
Dashboard update	< 2 s p95	Event arrival → UI state
Scenario simulation	< 5 s for small scenario	API timer

Metric	Target	Measurement
Recommendation generation	< 8 s including explanation	Trace full workflow
Availability during demo	≥ 99% over test window	Synthetic health checks

These are engineering targets, not claims of achieved performance. Benchmark results should be recorded from the final deployed build.

8.5 Optimization techniques

- Precompute static geospatial indexes and cache repeated spatial joins.
- Batch compatible model inference while preserving event-time semantics.
- Use asynchronous jobs for expensive scenario simulations.
- Cache invariant calculations across what-if runs.
- Use compact tabular models for low-latency decisions and reserve deep learning for tasks that benefit from it.
- Instrument every stage so optimization targets the actual bottleneck.

9. Conclusion & Impact Summary

SkyGuardX-AI is designed around a simple but ambitious proposition: **the most useful AI system for a critical event is not the one that produces the most sophisticated prediction; it is the one that turns a timely prediction into a defensible decision.**

The project combines space intelligence, Earth observation, streaming data, AI/ML, geospatial analytics, cascading impact modeling, scenario simulation and generative explanations. IBM Z is positioned as the secure, reliable transactional/data and inference foundation that brings intelligence closer to critical business and operational decisions.

9.1 Expected outcomes

Outcome	How SkyGuardX-AI demonstrates it
Faster response	Streaming event path and immediate risk/impact updates
Better decisions	Ranked alternatives rather than single alerts
Earlier intervention	Prediction and what-if analysis before projected impact
Explainability	Evidence-backed rationale and model metadata
Responsible AI	Human approval, policy constraints, confidence and audit
Technical ambition	Cross-domain cascade engine and counterfactual simulation
IBM Z relevance	Transactional AI/data processing positioned at the decision boundary

9.2 Final demo narrative

1. "A new space/remote-sensing event arrives."
↓
2. "AI detects elevated risk."
↓
3. "The system predicts the likely Earth impact."
↓
4. "The cascade engine identifies affected roads, population and critical facilities."
↓
5. "The operator asks: WHAT IF we act differently?"
↓
6. "The simulator compares response strategies."
↓
7. "SkyGuardX-AI recommends the lowest-risk feasible action."
↓
8. "The operator sees WHY, approves, and the decision is recorded for audit."

NORTH-STAR MESSAGE

From Space Signals to Critical Decisions: detect early, predict responsibly, understand cascading impact, simulate alternatives, and help humans act before consequences become irreversible.

Appendix A — Buildathon implementation checklist

- Canonical event schema implemented and validated
- Replay/simulation mode working
- At least one space intelligence model working
- At least one Earth hazard workflow working
- Geospatial exposure calculation working
- Cascade impact score visible

- ■ At least two what-if actions compared
- ■ Recommendation contains confidence + evidence
- ■ Human approval gate demonstrated
- ■ Audit record persisted
- ■ IBM Z integration/boundary demonstrated with a concrete data or inference flow
- ■ End-to-end demo completes without manual database edits

Appendix B — Recommended repository structure

```
nexus-z/
├── apps/
│   ├── web/           # React command center
│   └── api/           # FastAPI gateway
├── services/
│   ├── ingestion/
│   ├── space-risk/
│   ├── earth-risk/
│   ├── cascade-engine/
│   ├── scenario-engine/
│   └── decision-engine/
├── models/
│   ├── anomaly/
│   ├── hazard/
│   └── forecasting/
├── data/
│   ├── schemas/
│   └── samples/
├── infra/
│   ├── docker/
│   └── deployment/
├── tests/
└── docs/
    ├── architecture/
    ├── api/
    └── model-cards/
```